



Παράλληλος και Κατανεμημένος Υπολογισμός I

Παράλληλος υπολογισμός (συν.2)

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025



Μετρικές και ανάλυση

Μετρήσεις χρόνου

```
Main {  
    ...           // αρχικοποίηση, Tinit  
    time_elapsed = gettimeofday();  
  
    for (i=0; i<N; i++) // αρχή υπολογισμού, Tcomp  
        create/start Thread (i);  
  
    for ((i=0; i<N i++) // αναμονή τερματισμού. Tcomp  
        wait/join Thread (i);  
  
    time_elapsed = time_elapsed - gettimeofday();  
    print(time_elapsed);  
    ...           // εμφάνιση αποτελέσματος, Tfinal  
}
```

Συνήθως μετρούμε χρόνο στο main, γιατί αρχίζει πρώτο και τελειώνει τελευταίο.

Ανάλυση εκτέλεσης

- Ο χρόνος εκτέλεσης μπορεί να αναλυθεί σε

$$\mathbf{T_{total} = T_{init} + T_{comp} + T_{final}}$$

Συνήθως T_{init} , T_{final} είναι I/O-bound και T_{comp} είναι CPU-bound.

- Υπολογιστικά εντατικός (CPU-bound) υπολογισμός: εντατική χρήση CPU, σε κάθε αναφορά στη μνήμη αντιστοιχούν αρκετές πράξεις.
- Επικοινωνιακά εντατικός (I/O-bound) υπολογισμός: εντατική χρήση δίσκου, δικτύου, σε κάθε αναφορά στη μνήμη αντιστοιχούν σχετικά λίγες πράξεις.

Πειράματα και μετρήσεις 1

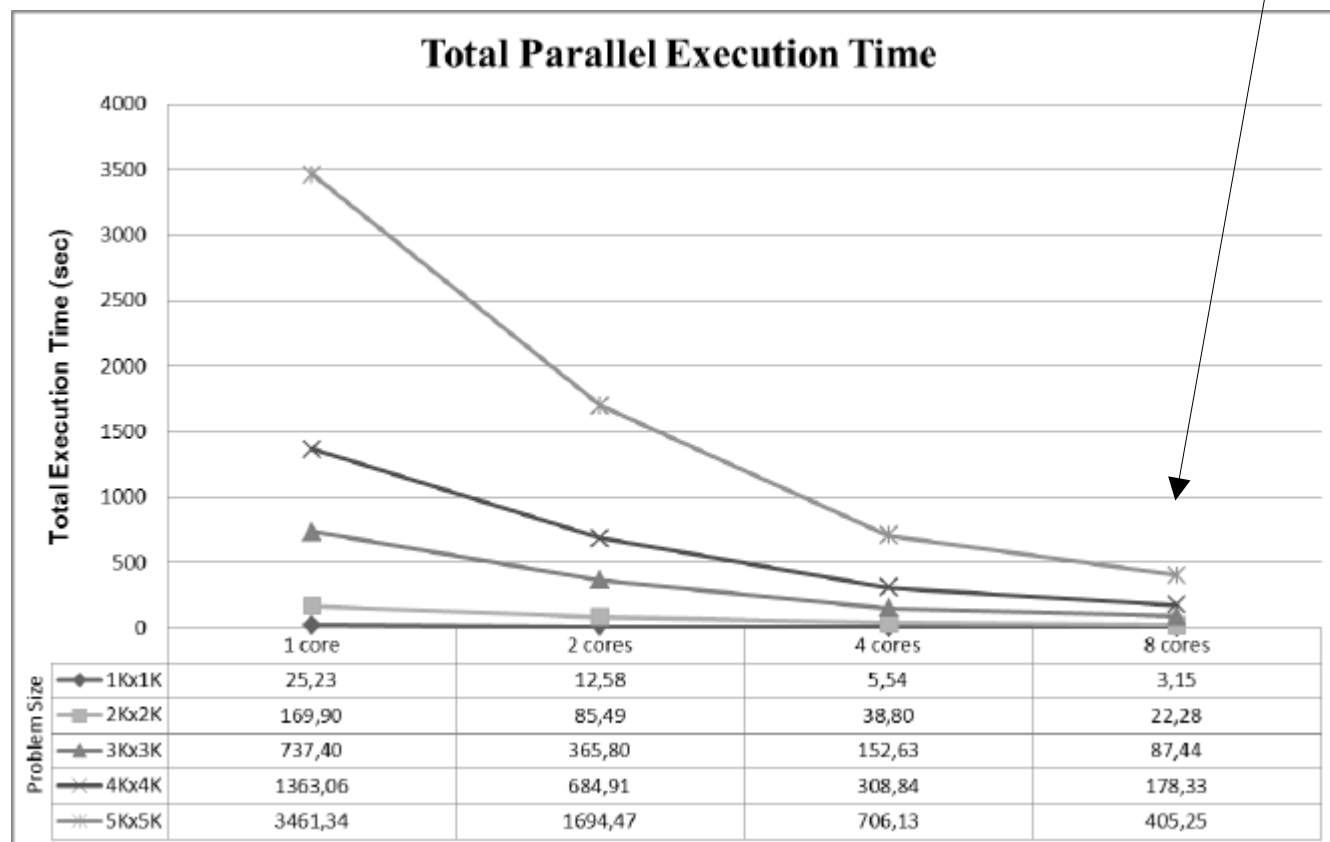
- Ξεκινούμε από δύο προγράμματα: το ακολουθιακό και το παράλληλο.
- Πολλές φορές αντί για καθαρά ακολουθιακό χρησιμοποιούμε ως σημείο αναφοράς το παράλληλο με 1 παράλληλη εργασία.
- Εκτελούμε πειράματα και μετρήσεις για μέγεθος προβλήματος $N1, N2, \dots$ και αριθμό παράλληλων εργασιών 1, 2, 4, 8 ... Καταγράφουμε τους χρόνους εκτέλεσης των εντατικών τμημάτων.
- Οι χρόνοι $T(*, 1)$ είναι χρήσιμοι για τον έλεγχο του ακολουθιακού κώδικα, αλγορίθμου. Τα $N1, N2, \dots$ έχουν μεταξύ τους σχέση $X2, X10, X1000, \dots$ ανάλογα με τον αλγόριθμο.

Πειράματα και μετρήσεις 2

N/P	1	2	...
N1	$T(N1,1)$	$T(N1,2)$	...
N2	$T(N2,1)$	$T(N2,2)$	...
...	...	...	...

Πιθανή αύξηση χρόνου
λόγω overheads

Τυπικό
(ιδανικό)
διάγραμμα



Πειράματα και μετρήσεις 3

- Υπολογίζουμε τους χρόνους εκτέλεσης για σχετικά μεγάλα N και με αρκετές (πχ 5) επαναλήψεις του ίδιου πειράματος.
- Αν οι χρόνοι εκτέλεσης είναι πολύ μικροί (πχ λίγα msec) η μέτρηση έχει πολύ “θόρυβο”. Μεγαλώνουμε το μέγεθος του προβλήματος.
- Αν οι χρόνοι δεν είναι ικανοποιητικοί τότε αναζητούμε πιθανές αιτίες στον κώδικα, στον αλγόριθμο, στο σύστημα χρόνου εκτέλεσης ή στο παράλληλο σύστημα.
- Κάνουμε διορθώσεις και επαναλαμβάνουμε τα πειράματα και τις μετρήσεις.

Πειράματα και μετρήσεις 4

- Οι ακολουθιακοί χρόνοι $T(N1,1)$, $T(N2,1)$... πρέπει να ακολουθούν ικανοποιητικά την πολυπλοκότητα του ακολουθιακού αλγορίθμου. Αν όχι... έχουμε κάποιο σφάλμα ή πολύ μικρό μέγεθος προβλήματος.
- Γενικά ένας ικανοποιητικός παράλληλος αλγόριθμος πρέπει να έχει καλύτερες επιδόσεις για μεγαλύτερα μεγέθη του προβλήματος. Δηλαδή οι χρόνοι για 2, 4, 8,.. εργασίες (επεξεργαστές) πρέπει να εμφανίζουν μεγαλύτερη μείωση για μεγαλύτερο μέγεθος προβλήματος.
- Όταν οι εργασίες είναι πολύ ελαφριές ή περισσότερες από τους επεξεργαστές μπορεί να έχουμε αυξήσεις στους χρόνους.

Επικοινωνία επεξεργαστή και μνήμης

- Αρκετά συχνά οι μετρήσεις χρόνου δεν είναι ικανοποιητικές ακόμη και όταν ο αλγόριθμος είναι σωστός.
- Ένας σημαντικός παράγοντας είναι η επικοινωνία επεξεργαστή μνήμης.
- Παρ' όλη την ιεραρχία κρυφής μνήμης, όταν έχουμε απλούς υπολογισμούς που σε κάθε βήμα απαιτούν πολλαπλές αναφορές στη μνήμη, πχ

$$c[i] = a[i] + b[i]$$

ο υπολογισμός καθυστερεί εξ αιτίας των περιορισμών στο εύρος ζώνης της επικοινωνίας επεξεργαστή μνήμης.

Βαρύτητα εργασιών: Αριθμητική πυκνότητα 1

- Αριθμητική Πυκνότητα =

Πλήθος πράξεων / πλήθος δεδομένων =

Υπολογισμός / Επικοινωνία

Ως επικοινωνία εννοούμε όχι μόνο την δικτυακή επικοινωνία αλλά και την μεταφορά δεδομένων από την μνήμη στη CPU. (bytes/sec)

Γενικά το εύρος ζώνης της επικοινωνίας είναι σημαντικά μικρότερο από την διεκπεραιωτική ικανότητα των επεξεργαστών (πράξεις/sec).

Επομένως θέλουμε να έχουμε όσο το δυνατό περισσότερους υπολογισμούς ανά δεδομένο.

Βαρύτητα εργασιών: Αριθμητική πυκνότητα 2

Δύο κριτήρια:

- Ο υπολογισμός πρέπει να είναι σημαντικός πχ αρκετές επαναλήψεις.
- Αριθμός δεδομένων ανά υπολογισμό. Ιδανικά θέλουμε πολλούς υπολογισμούς ανά load, store.

Για παράδειγμα:

$a[i] = b[i] + c[i]$: 1 πράξη / 3 load, store

$a[i] = s * a[i]$: 1 πράξη / 2 load, store

$a[i] = \text{mysinx}(a[i])$: 500 πράξεις / 2 load, store

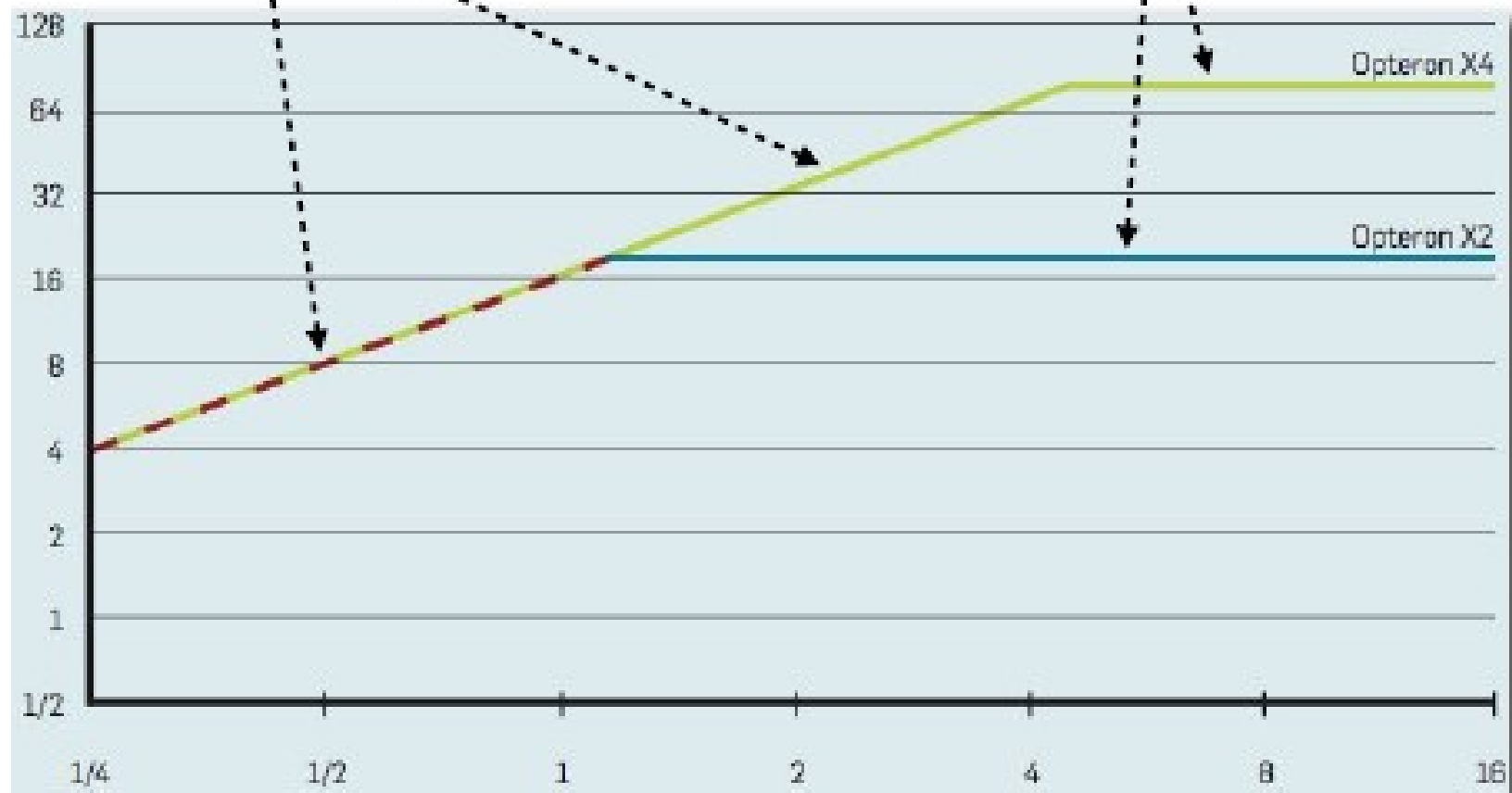
$a[i] = \text{mysqrt}(a[i])$: ... / 2 load, store

Μοντέλο Οροφής (Roofline model)

Περιοχή που η επικοινωνία με τη μνήμη περιορίζει τις επιδόσεις

Περιοχή που οι επιδόσεις εξαρτώνται από την επεξεργαστική ισχύ

Εύρος Ζώνης



Αριθμητική πυκνότητα υπολογισμού

Αναζήτηση πιθανών αιτιών προβλημάτων 1

- Πρόσθεση πράξεων χωρίς αλλαγή στις προσβάσεις στη μνήμη (πχ “μεγαλύτερο” for loop $a[i] = \text{mysin}$).

Αν ο χρόνος εκτέλεσης αυξάνει γραμμικά με την αύξηση των επαναλήψεων στο for τότε φθάσαμε στο μέγιστο της ισχύος του επεξεργαστή μας (στην οριζόντια γραμμή του μοντέλου οροφής).

- Αφαίρεση όλων των πράξεων χωρίς αλλαγή στις προσβάσεις στη μνήμη (πχ “αδειος” κώδικας $a[i] = 0$).

Αν ο χρόνος εκτέλεσης δεν μειώνεται σημαντικά σε σχέση με τον κανονικό υπολογισμό τότε βρισκόμαστε στη κεκλιμένη γραμμή του μοντέλου οροφής.

Αναζήτηση πιθανών αιτιών προβλημάτων 2

- Μετατρέπουμε τις προσβάσεις μνήμης σε $a[0]$ δηλαδή έχουμε πλήρη τοπικότητα.

Αν ο χρόνος εκτέλεσης μειώνεται σημαντικά τότε έχουμε πρόβλημα στη τοπικότητα στις προσπελάσεις της μνήμης.

- Αφαίρεση μηχανισμών ταυτοχρονισμού.

Αν ο χρόνος εκτέλεσης μειώνεται σημαντικά τότε έχουμε προβλήματα υπερβολικής χρήσης μηχανισμών ταυτοχρονισμού.

Αναμενόμενη Επιτάχυνση

- Η επιτάχυνση είναι μια μετρική εκτίμησης της επίδοσης του παράλληλου υπολογισμού.
- Πρώτα κάνουμε μια εκτίμηση της αναμενόμενης επιτάχυνσης:

Αναμενόμενη επιτάχυνση =

Βήματα εκτέλεσης του ακολουθιακού προγράμματος για μέγεθος προβλήματος N

Βήματα εκτέλεσης παράλληλου προγράμματος για μέγεθος προβλήματος N με P εργασίες (σε P επεξεργαστές).

(όπου βήματα εκτέλεσης πρόγραμμα θα μπορούσαμε να βάλουμε πολυπλοκότητα αλγορίθμου)

Παρατηρούμενη Επιτάχυνση 1

Παρατηρούμενη επιτάχυνση =

Χρόνος εκτέλεσης του ακολουθιακού προγράμματος για μέγεθος προβλήματος N : $T(N, 1)$

Χρόνος εκτέλεσης παράλληλου προγράμματος για μέγεθος προβλήματος N με P εργασίες: $T(N, P)$

Speedup $S(N, P) = T(N, 1) / T(N, P)$

Θέλουμε $T(N, P) \sim T(N, 1)/P$ δηλαδή $S(N, P) \sim P$

Παρατηρούμενη Επιτάχυνση 2

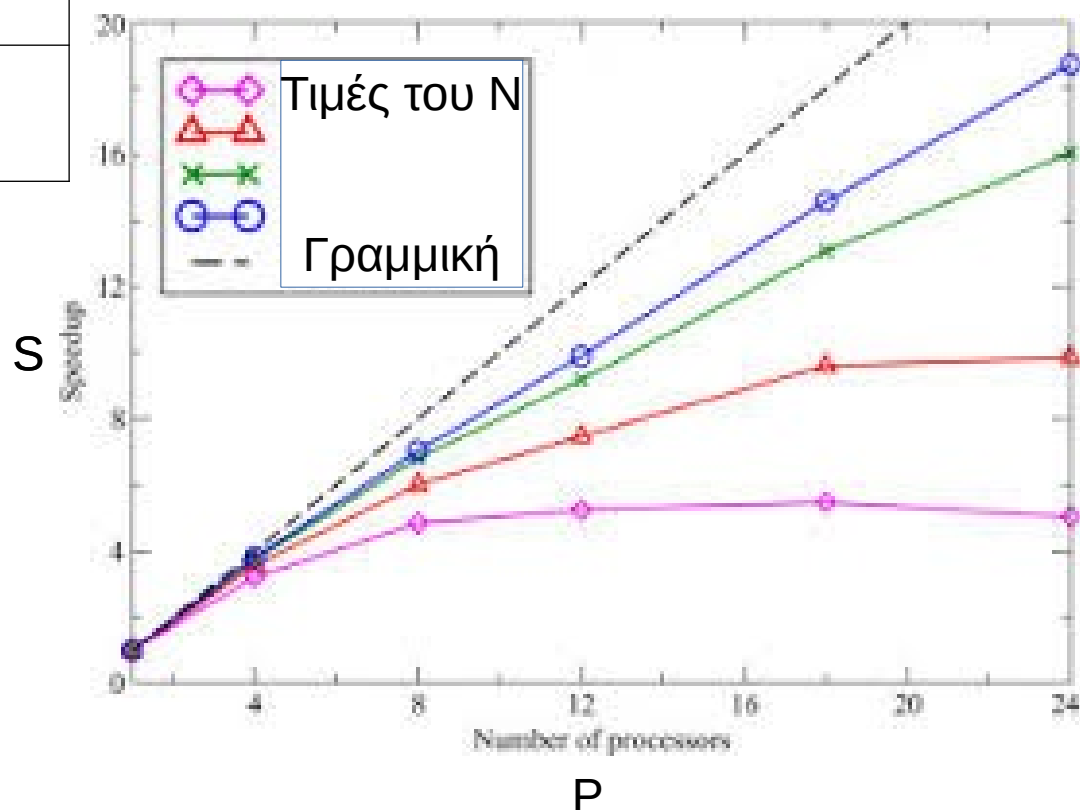
N/P	1	2	...
N1	$T(N1,1)/T(N1,1)$	$T(N1,1)/T(N1,2)$	...
N2	$T(N2,1)/T(N2,1)$	$T(N2,1)/T(N2,2)$	...
...	...	...	...

$S(N,P)$

Τυπικό
διάγραμμα

Γενικά ισχύει $T(N, P) \geq T(N, 1) / P$

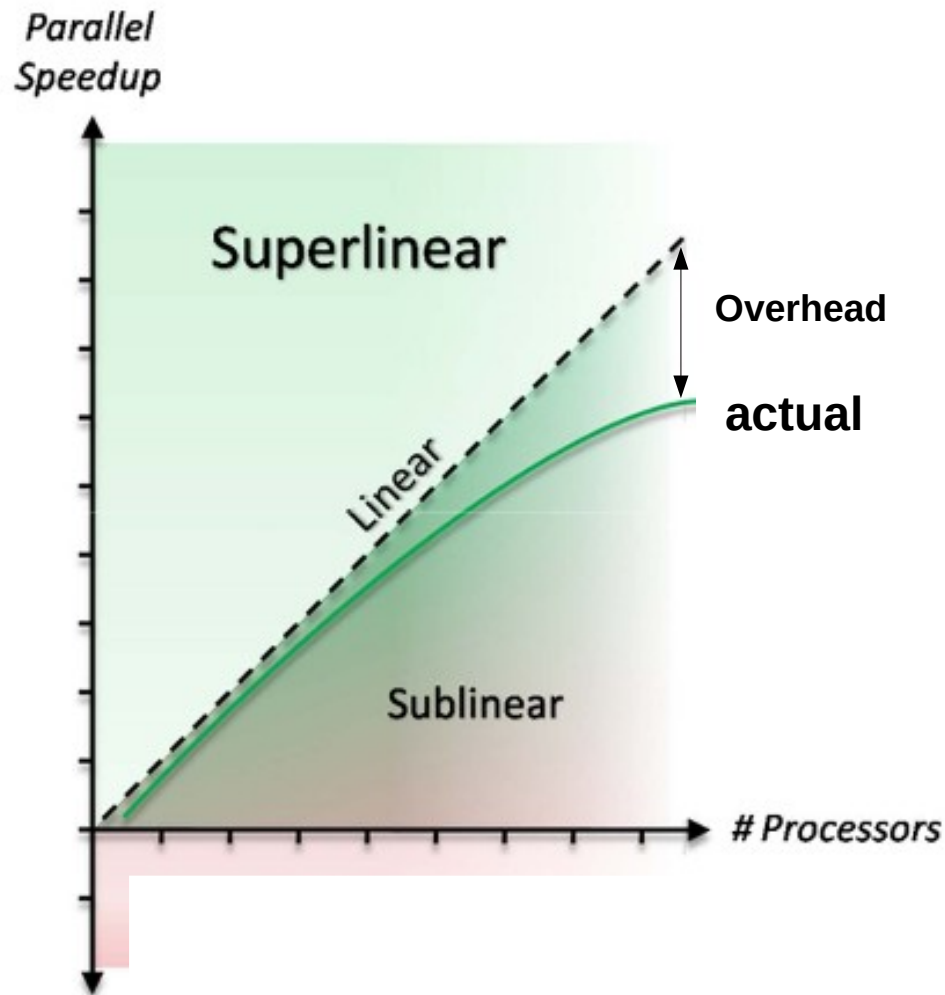
Γραμμική επιτάχυνση έχουμε όταν $T(N, P) = T(N,1) / P$, άρα $S = P$.



Επιτάχυνση 1

- Όλες οι μετρήσεις αναφέρονται στο ίδιο σύστημα / μεταγωγτιστή / βελτιστοποιήσεις / αλγόριθμο. Δηλαδή δεν αλλάζουμε τα χαρακτηριστικά εκτέλεσης μεταξύ των εκτελέσεων, μόνο τον αριθμό των εργασιών P ή και το μέγεθος του προβλήματος N .
- **Σχετική** (relative) επιτάχυνση: αντί για ακολουθιακό κώδικα, εκτελούμε παράλληλο κώδικα με 1 εργασία.
- **Απόλυτη** (absolute) επιτάχυνση: σύγκριση ακολουθιακού με παράλληλο κώδικα.

Επιτάχυνση 2



- Γραμμική (linear) επιτάχυνση: όταν η προσθήκη επεξεργαστών προκαλεί ευθέως ανάλογη αύξηση της επιτάχυνσης.
- Πραγματική (actual) επιτάχυνση: αυτή που μετράμε με τα πειράματα.
- Επιβάρυνση (overhead): η διαφορά γραμμικής και πραγματικής επιτάχυνσης.
- Επιτάχυνση μικρότερη της μονάδας: ο παράλληλος αλγόριθμος είναι πιο αργός από τον ακολουθιακό.
- Υπεργραμμική (superlinear) επιτάχυνση: μάλλον κάτι ύποπτο συμβαίνει :-)

Επβαρύνσεις (Overheads) 1

Προέρχονται:

- Από το παράλληλο σύστημα, πχ. πρόσβαση στην ιεραρχία μνήμης (πολύ σημαντική παράμετρος).
- Από το λειτουργικό σύστημα, πχ διαχείριση νημάτων.
- Από τη γλώσσα, πχ αδυναμίες του μεταγλωττιστή / διερμηνευτή και του συστήματος χρόνου εκτέλεσης.
- Από την υλοποίηση (το πρόγραμμα), πχ προγραμματιστικές αδυναμίες, πχ αριθμητική πυκνότητα.
- Από τον παράλληλο αλγόριθμο, πχ αλγόριθμοι με εξαρτήσεις ή καταστάσεις.

Επιβαρύνσεις 2

Σχέση φόρτου εργασίας και κόστους διαχείρισης:

- Όσο περισσότερα νήματα τόσο μικρότερος ο φόρτος εργασίας ανά νήμα.
- Όσο περισσότερα νήματα τόσο μεγαλύτερη η επιβάρυνση διαχείρισης των νημάτων.
- Αν η απαιτούμενη εργασία είναι πολύ μικρή τότε πιθανώς η επιβάρυνση διαχείρισης υπερσχύει του κέρδους από τον παραλληλισμό.
- Ο ακριβείς τιμές για το 'φόρτο εργασίας' και την επιβάρυνση εξαρτώνται από το σύστημα και τη γλώσσα προγραμματισμού.

Κόστος και αποδοτικότητα

- **Κόστος :**

$$\text{Cost (N, P)} = T(N, P) * P$$

$$\text{Cost (N, 1)} = T(N, 1) * 1 = T(N, 1)$$

- **Αποδοτικότητα :** Κόστος(1) / Κόστος (P)

Δίνει το ποσοστό ωφέλιμης χρήσης των επεξεργαστών

$$\text{Efficiency (N, P)} = \text{Cost (N, 1)} / \text{Cost (N, P)}$$

$$= T(N, 1) / T(N, P) * P$$

$$= S(N, P) / P$$

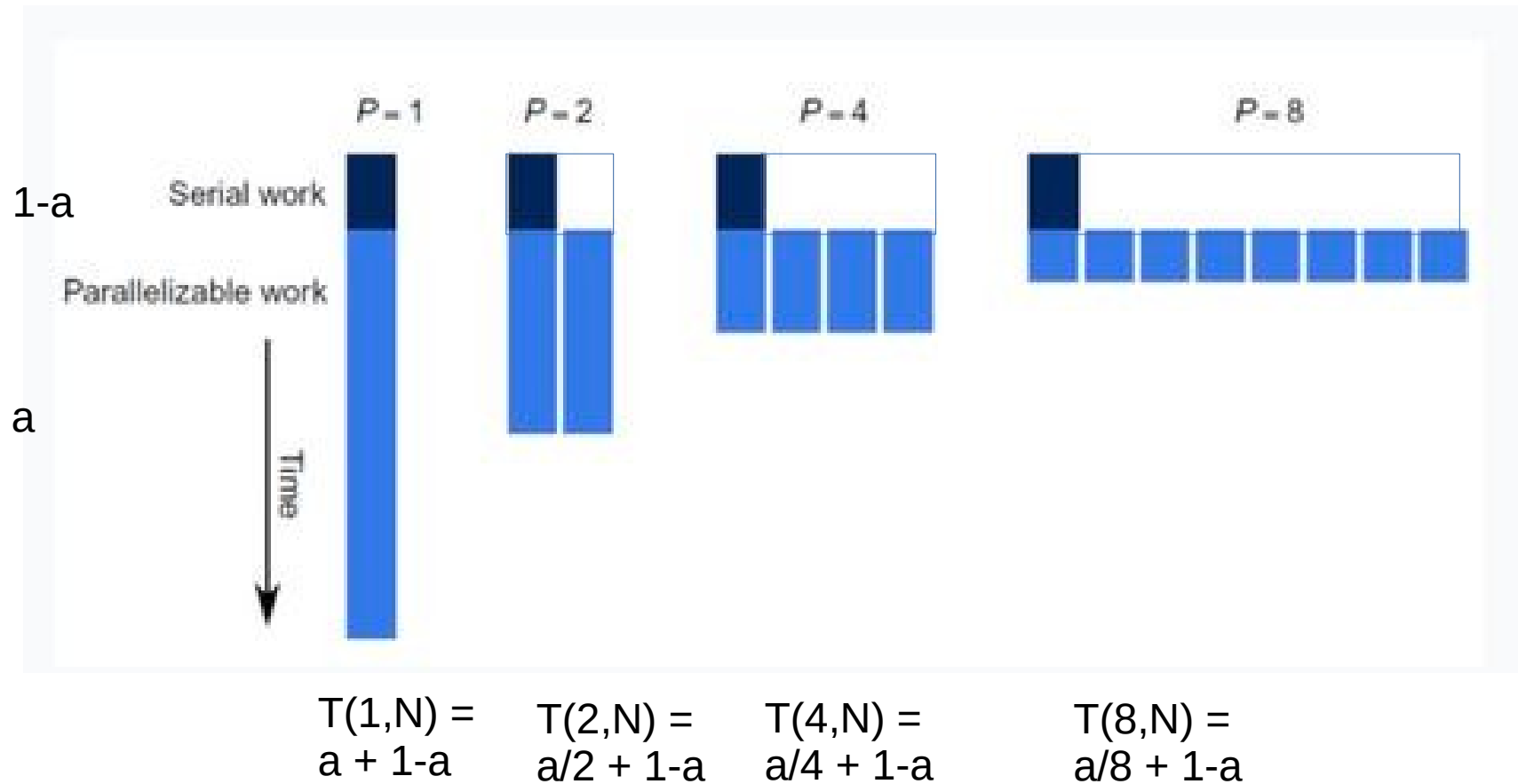
Επιτάχυνση και κόστος

- Ένας παράλληλος αλγόριθμος μπορεί να επιτύχει καλύτερη επιτάχυνση αλλά να χρησιμοποιεί μεγαλύτερο αριθμό επεξεργαστών.
- Ο αλγόριθμος δεν είναι cost-optimal, γιατί έχει μεγάλο κόστος (ή μικρή αποδοτικότητα).
- Ένας cost-optimal παράλληλος αλγόριθμος μπορεί να μην είναι απαραίτητα ο ταχύτερος αλλά πετυχαίνει βέλτιστη αποδοτικότητα.

Νόμος Amdahl 1

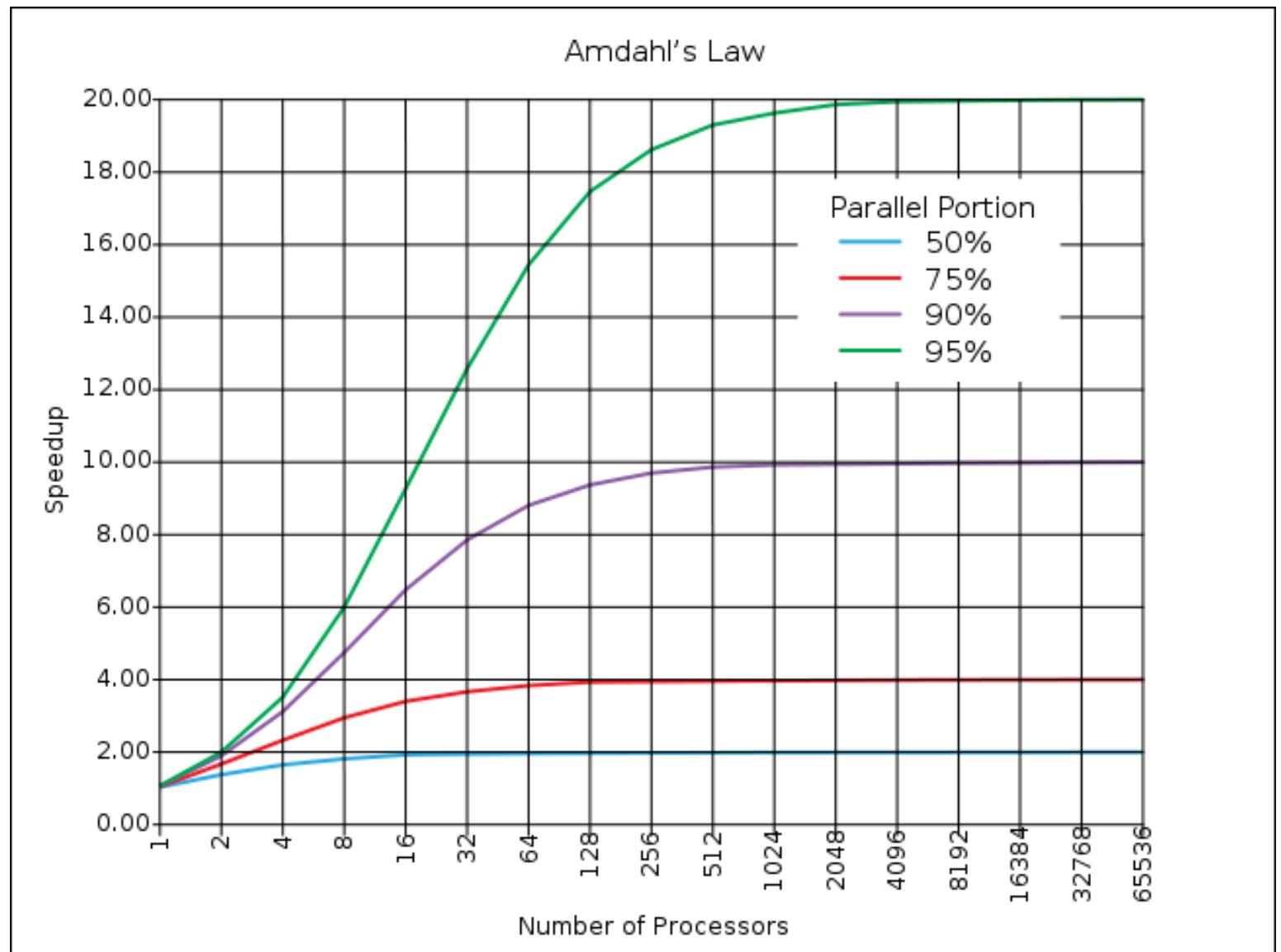
- Έστω $T(N, 1) = 1 = a + (1-a)$
όπου $0 \leq a \leq 1$ το παραλληλοποιήσιμο τμήμα και
 $1 - a$ το μη-παραλληλοποιήσιμο τμήμα του κώδικα.
- Τότε $T(N, P) = a / P + (1-a)$
παραλληλοποιείται μόνο το τμήμα a .
- $$S(N,P) = T(N,1) / T(N,P) = [a + (1-a)] / [a / P + (1-a)]$$
$$= 1 / [a / P + (1-a)]$$
- Για $P \rightarrow \infty$, $a / P \rightarrow 0$, άρα $S(N,\infty) \rightarrow 1 / (1-a)$

Νόμος Amdahl 2



Μελετήστε Χρόνο, Επιτάχυνση, Κόστος και Αποδοτικότητα.

Νόμος Amdahl 3



Η προσθήκη επεξεργαστών πέρα από το όριο που καθορίζει το μη-παραλληλοποιήσιμο τμήμα του κώδικα δεν επιφέρει βελτίωση της επιτάχυνσης.

Παρατηρήσεις 1

- Δεν υπάρχει λόγος το σύνολο του αλγορίθμου ή κώδικα να αντιμετωπίζεται σαν μια ενότητα.
- Διαφορετικά τμήματα κώδικα επιδέχονται διαφορετικού τύπου ή και καθόλου παραλληλισμό.
- Για τα διαφορετικά τμήματα μπορεί να χρησιμοποιηθούν τα κατάλληλα υπολογιστικά συστήματα.
- Τυπικό παράδειγμα οι μονάδες επεξεργασίας γραφικών που λειτουργούν ως συνεπεξεργαστές.

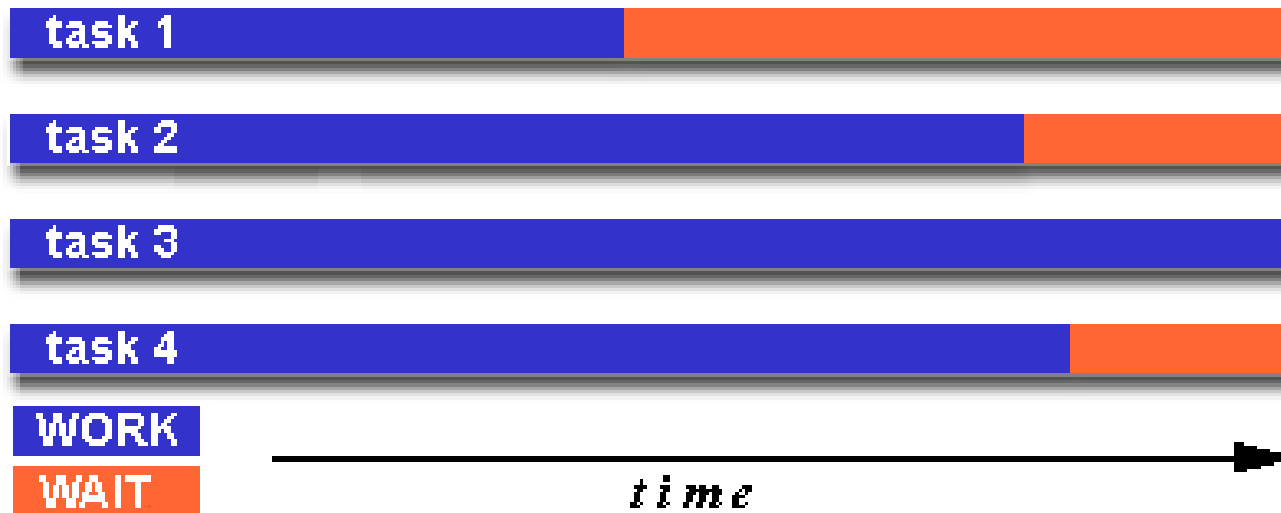
Παρατηρήσεις 2

- Δεν παραλληλίζουμε γενικά τον κώδικα αλλά εστιάζουμε σε περιοχές που επιβαρύνουν την εκτέλεση.
- Δεν είναι όλοι οι αλγόριθμοι παραλληλοποιήσιμοι με εξίσου ικανοποιητικά αποτελέσματα.
- Ένας ακολουθιακός αλγόριθμος μπορεί να παραλληλοποιηθεί με διαφορετικούς τρόπους.
- Η παραλληλοποίηση ενός ακολουθιακού αλγορίθμου δεν αλλάζει τη πολυπλοκότητά του, αλλά μειώνει το χρόνο εκτέλεσης σε ‘αποδεκτά’ όρια.



Εξισορρόπηση υπολογιστικού φόρτου

Εξισορρόπηση υπολογιστικού φόρτου



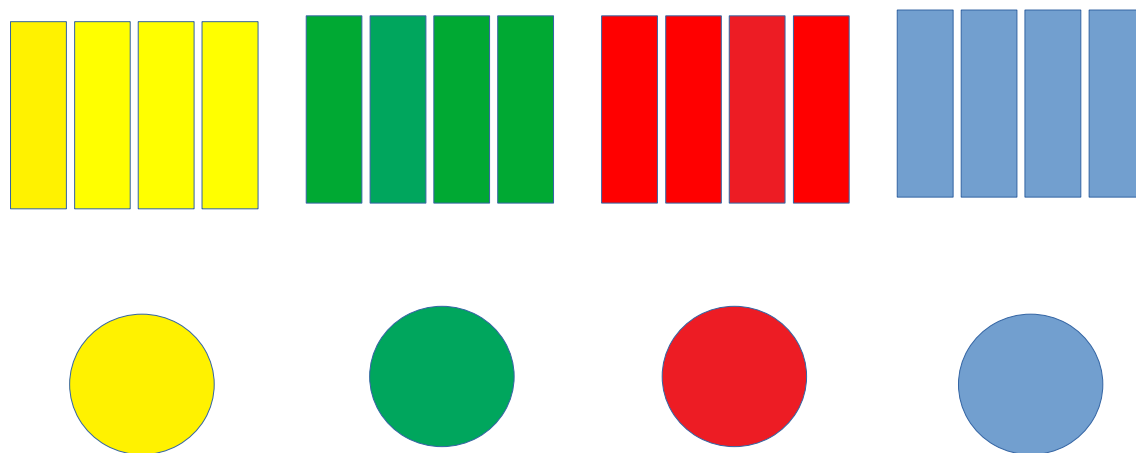
- Για να ελαχιστοποιηθεί ο χρόνος εκτέλεσης οι παράλληλες εργασίες πρέπει να εκτελούν περίπου ίσο υπολογιστικό έργο, ή ο υπολογιστικός φόρτος να είναι ισοκατανεμημένος.

Code/LoadBalancing

Γενίκευση της στατικής κατανομής 1

Έστω n δεδομένα/επαναλήψεις και P εργασίες.

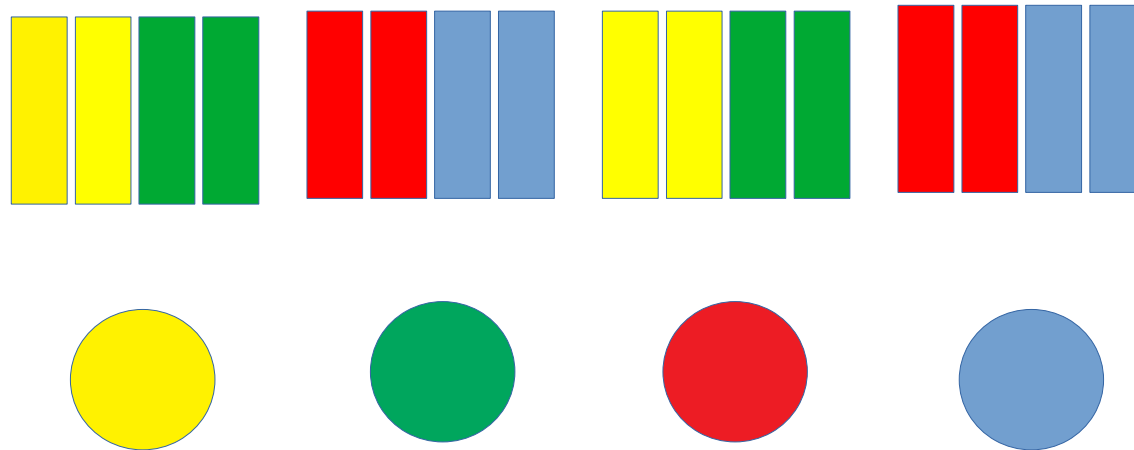
- Μέχρι τώρα υποθέταμε ότι κάθε υπολογισμός (ανά δεδομένο ή ανά επανάληψη) διαρκεί ίσο χρόνο με τους υπολοίπους.
- Τα δεδομένα (ή οι επαναλήψεις) κατανέμονται σε ένα κύκλο των P blocks με n/P στοιχεία σε κάθε block (blockSize).



Στατική κατανομή: $n=16$, $P=4$, $\text{blockSize}=n/P$

Γενίκευση της στατικής κατανομής 2: Block κυκλική κατανομή

- Επέκταση σε μεγαλύτερο πλήθος blocks. Για παράδειγμα, θα μπορούσαμε να έχουμε $n\text{Blocks} = 2P$ blocks με $\text{blockSize} = n/2P$.
- Τα δεδομένα/επαναλήψεις κατανέμονται σε 2 κύκλους, πρώτα P blocks και μετά άλλα P blocks μεγέθους $n/2P$.



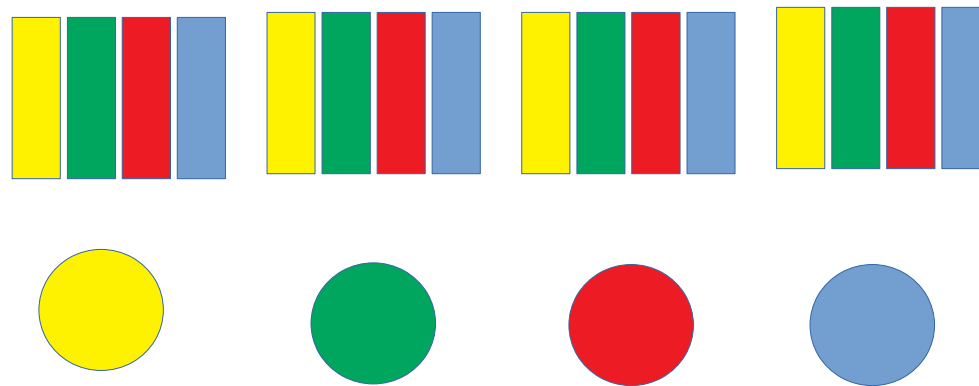
Κατανομή με $n=16$, $P=4$, $n\text{Blocks}=2P$, $\text{blockSize}=n/2P$

Γενίκευση της στατικής κατανομής 3: Απλή κυκλική κατανομή

- Με την ίδια λογική μπορούμε να έχουμε:

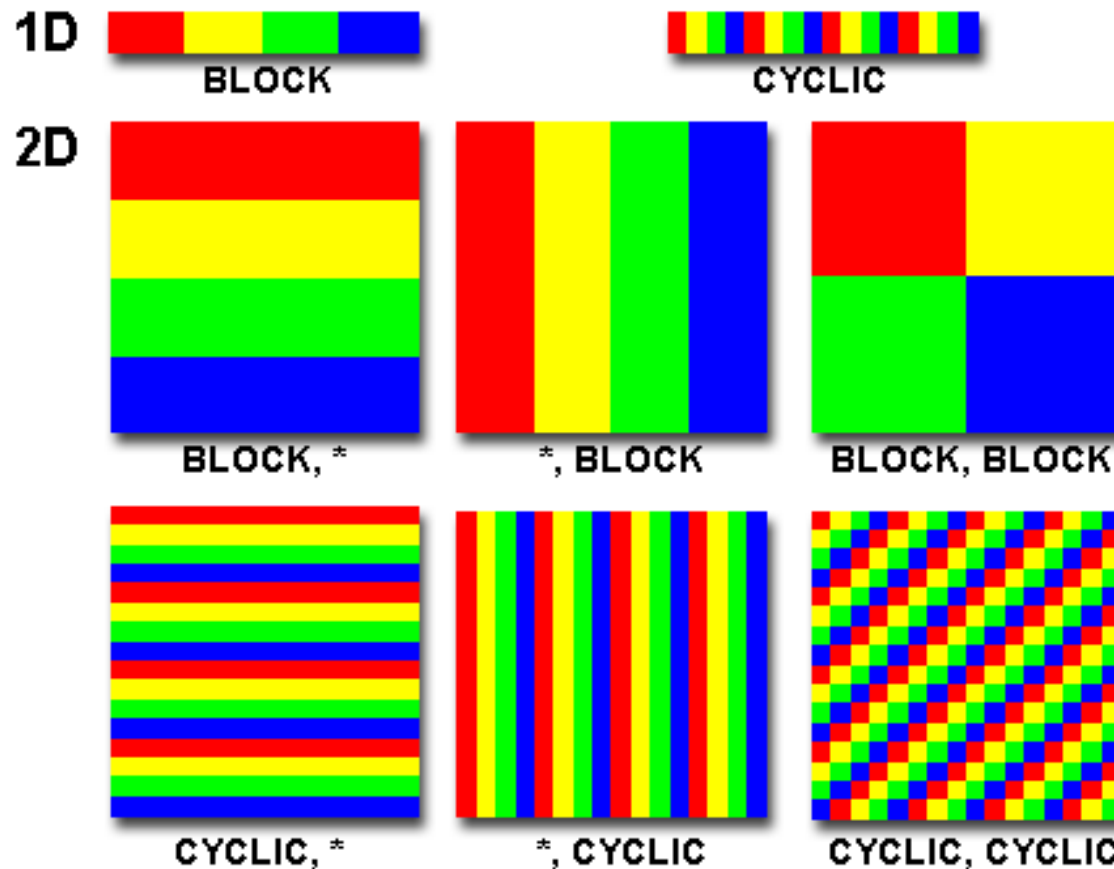
4 κύκλους κατανομής με $4P$ blocks μεγέθους $n/4P$
8 κύκλους κατανομής με $8P$ blocks μεγέθους $n/8P$
- Στην ακραία περίπτωση μπορεί να έχουμε

 n/P κύκλους κατανομής με n blocks μεγέθους 1.



Κατανομή με $n=16$, $P=4$, $n/P=4$, $nBlocks=n$, $blockSize=1$

Στατική και κυκλική κατανομή



Στατική, κυκλική και block κυκλική κατανομή

Όλες είναι μορφές στατικής κατανομής.

Από την σχεδίαση είναι γνωστό ποια δεδομένα θα αναλάβει κάθε εργασία.

Στατική κατανομή

shared $x[n]$, P

main:

P parallel tasks, $t=0,1,2,\dots,P-1$

task(i) {

$block = n / P;$

$start = i * block;$

$stop = start + block;$

 if ($i == P-1$) $stop = n;$

 for ($j = start; j < stop; j++$)

$x[j] = f(x[j]);$

}



Απλή κυκλική κατανομή:
 $n\text{Blocks} = n$, $\text{blockSize} = 1$

shared $x[n]$, P ;

main:

P parallel tasks, $t=0,1,2,\dots,P-1$

```
task(t) {  
    for (i=t; i<n; i=i+P)  
        f(x[i]);  
}
```

Block κυκλική κατανομή

```
shared x[n], P, blockSize, nBlocks=n/blockSize;  
// υποθέτουμε ότι  $n \% blockSize = 0$ 
```

```
main:
```

```
    P parallel tasks,  $t=0,1,2,\dots,P-1$ 
```

```
task(t) {  
    for (i=t; i<nBlocks; i=i+P) {  
        blockStart = i * blockSize;  
        blockStop = blockStart + blockSize;  
        for (j=blockStart; j<blockStop; j++)  
            f(x[j]);  
    }  
}
```

Πότε είναι χρήσιμη η κυκλική κατανομή

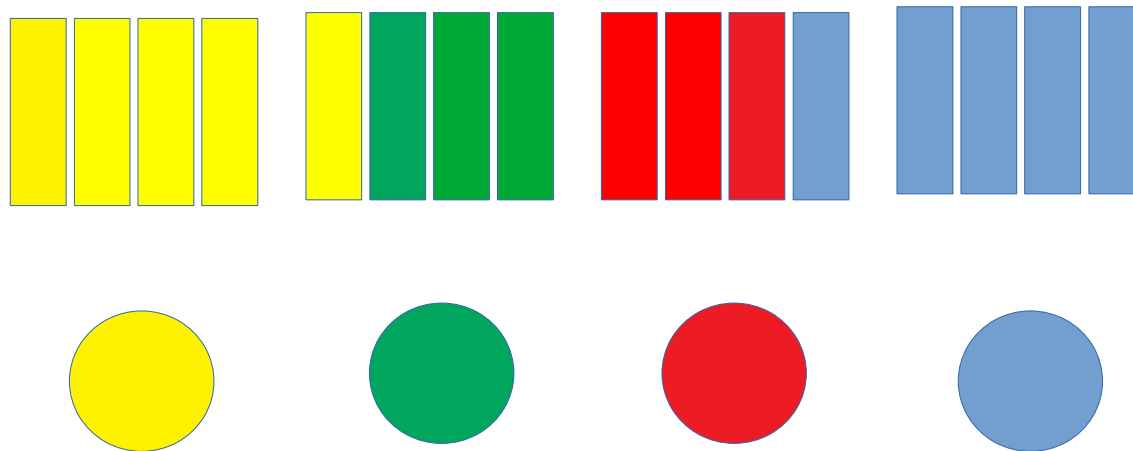
- Όταν γνωρίζουμε από την αρχή ότι οι υπολογισμοί (ανά δεδομένο ή επανάληψη) απαιτούν διαφορετικό χρόνο εκτέλεσης, όμως οι διαφορές κυμαίνονται μέσα σε κάποιο γνωστό εύρος
- Επίσης μπορεί να γνωρίζουμε την σχετική κατανομή των ‘βαρύτερων’ υπολογισμών μέσα στο σύνολο των υπολογισμών.
- Προσπαθούμε να δημιουργήσουμε αρκετά blocks ώστε να διασφαλίσουμε ότι οι ‘βαριοί’ υπολογισμοί δεν θα εκτελεστούν όλες από λίγες εργασίες (άρα λίγους επεξεργαστές).

Ζητήματα με την κυκλική κατανομή

- Πολύ μικρά `blockSize` σημαίνει πολλά blocks, μεγαλύτερη πιθανότητα για εξισορρόπηση φορτίου.
- Το `blockSize` πρέπει να είναι αρκετά μεγάλο ώστε να καλύπτει τουλάχιστο μια γραμμή κρυφής μνήμης, αλλιώς μπορεί να έχουμε υπερβολικά πολλές προσπελάσεις στη μνήμη για λίγα δεδομένα.
- Γενικά απαιτείται πειραματισμός για να βρούμε το σωστό `blockSize`.

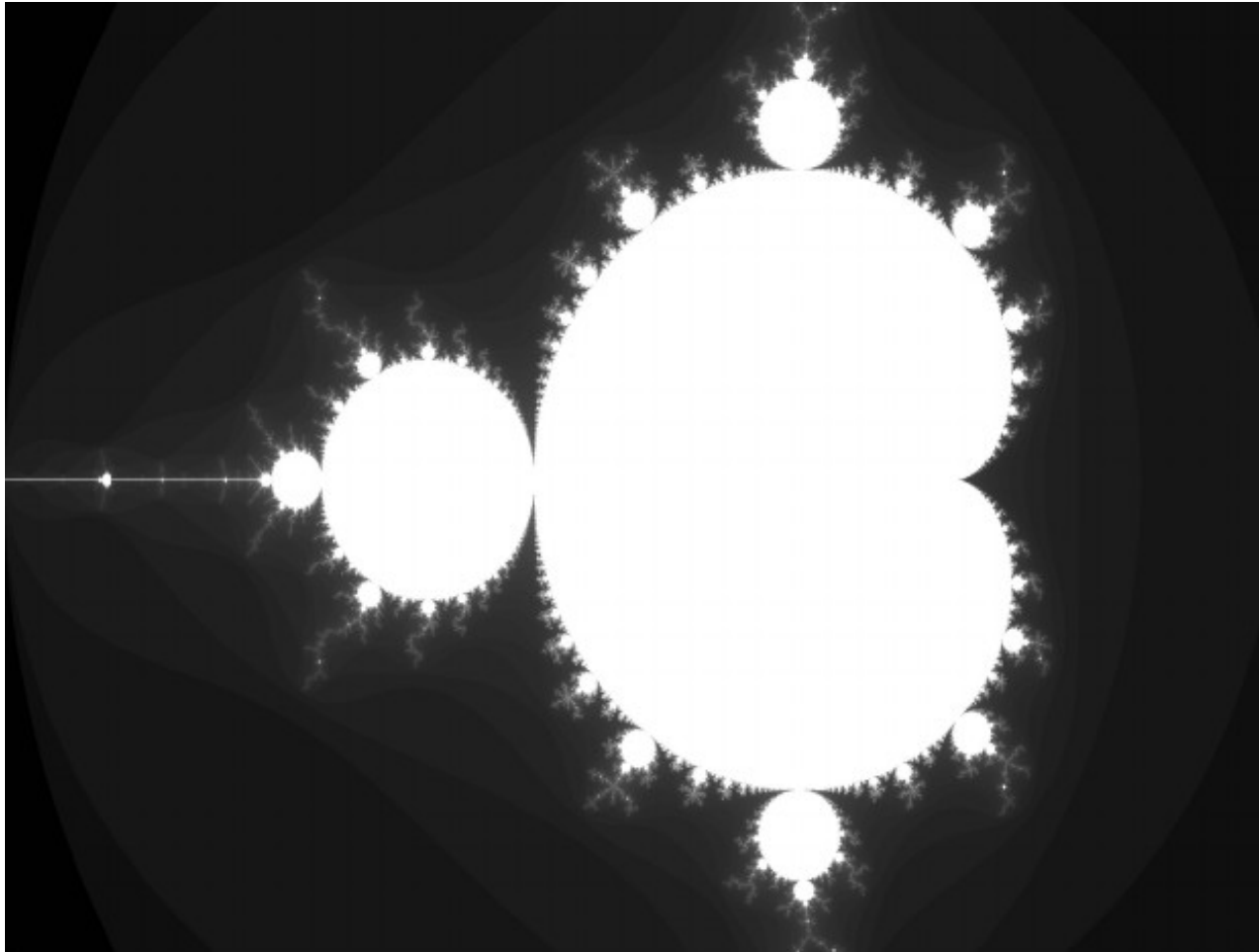
Γενίκευση της στατικής κατανομής 4: Προσαρμοσμένη στατική κατανομή

- Με γνωστό το υπολογιστικό κόστος κάθε υπολογισμού μπορούμε να προσαρμόσουμε τα δεδομένα (ή τις επαναλήψεις) ώστε να επιτύχουμε εξισορρόπηση.
- Αυτή η μέθοδος απαιτεί εξειδικευμένη εφαρμογή για κάθε πρόβλημα, δύσκολη γενίκευση.



Προσαρμοσμένη στατική κατανομή: $n=16$, $P=4$, δύο εργασίες (πράσινη, κόκκινη) λαμβάνουν εκτελούν λιγότερους αλλά πιο απαιτητικούς υπολογισμούς.

Σύνολο του Mandelbrot 1



Μαύρο χρώμα: σύντομος υπολογισμός.

Λευκό χρώμα: δαπανηρός υπολογισμός.

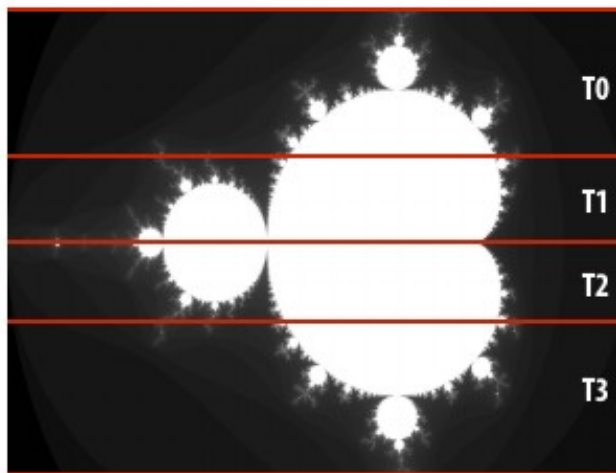
https://en.wikipedia.org/wiki/Mandelbrot_set
Code/Mandelbrot

Σύνολο του Mandelbrot 2

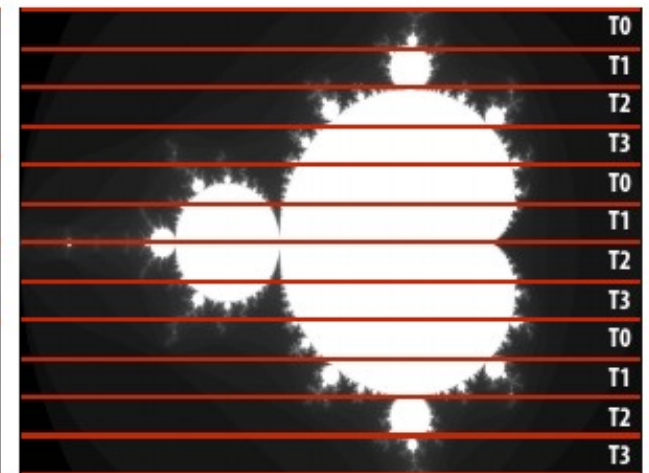
Πιθανές λύσεις



1



2



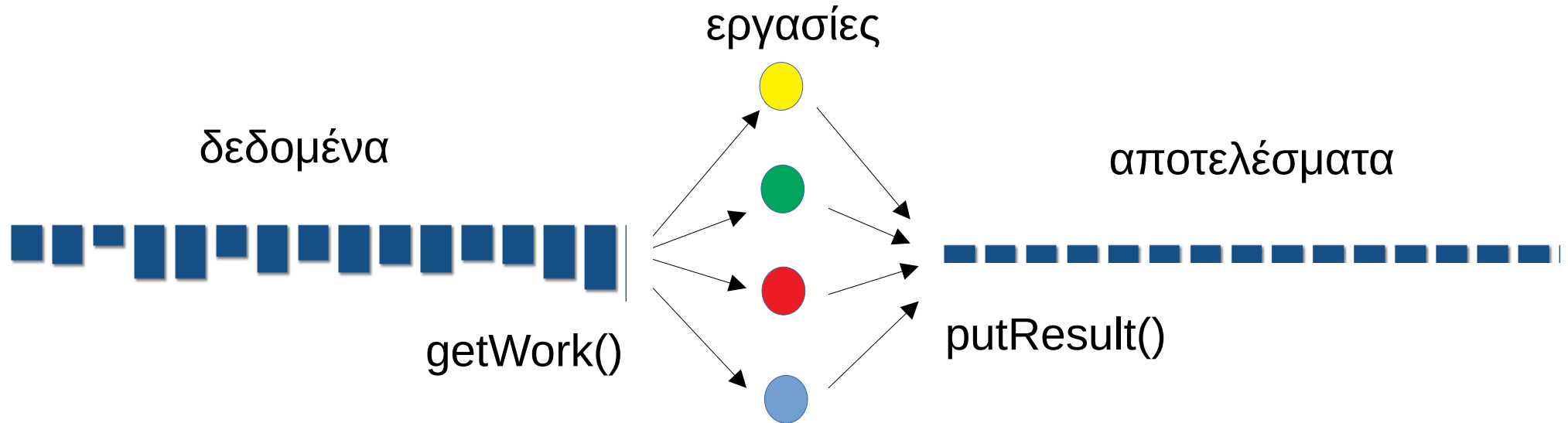
3

1. Στατική κατανομή (ανισορροπία φορτίου)
2. Προσαρμοσμένη στατική κατανομή (μάλλον δύσκολο)
3. Απλή κυκλική κατανομή (υπερβολικό κόστος συγχρονισμού)
3. Block κυκλική κατανομή (πρέπει να βρούμε το κατάλληλο block)

Δυναμική κατανομή

- Η κατανομή των δεδομένων (ή των επαναλήψεων) καθορίζεται κατά την εκτέλεση του παράλληλου προγράμματος.
- Οι εργασίες που εκτελούν δαπανηρούς υπολογισμούς λαμβάνουν λιγότερα δεδομένα.
- Το σύστημα προσαρμόζεται δυναμικά, δεν χρειάζεται να σχεδιαστεί προσαρμοσμένη στατική κατανομή.
- Χρήσιμη όταν ο υπολογιστικός φόρτος εξαρτάται από τα εκάστοτε δεδομένα (πχ διαφορετικά γραφήματα) και δεν είναι από την αρχή γνωστός αλλά διαφέρει σε κάθε στιγμιότυπο του προβλήματος.

Δυναμική κατανομή



- Τα δεδομένα μπορεί να είναι οργανωμένα σε ταυτόχρονη ουρά. Το `getWork()` μπορεί να ισοδυναμεί με `dequeue()`.
- Τα αποτελέσματα μπορούν να αποθηκευτούν στην ίδια ή σε άλλη ουρά με `putResult()` που μπορεί να ισοδυναμεί με `enqueue()`.

Απλή δυναμική κατανομή 1

- Κάθε εργασία λαμβάνει ένα στοιχείο των δεδομένων (ή μια επανάληψη). Όταν επιστρέφει το αποτέλεσμα τότε λαμβάνει νέο υπολογισμό.
- Επέκταση στην επεξεργασία block δεδομένων (ή επαναλήψεων) μεγέθους blockSize το καθένα.

Απλή δυναμική κατανομή 2

```
shared input[n], output[n], P,  
      index_in = 0, index_out = 0;  
lock_type l_in, l_out;
```

```
main :  
    P parallel tasks, t=0,1,2,...P-1
```

```
task(t) {  
    while (my_index = getWork()) > -1)  
        putResult( f(input[my_index]) );  
}
```

Απλή δυναμική κατανομή 3

```
int getWork() {  
    Lock(l_in);  
    if (index_in > -1 && index_in < n-1)  
        index_in++;    // input queue not empty  
    else  
        index_in = -1; // input queue empty  
    Unlock(l_in)  
    return index_in;  
}  
  
PutResult(item) {    // output queue has n locations  
    Lock(l_out);  
    output[index_out++] = item;  
    Unlock(l_out);  
}
```

Απλή δυναμική κατανομή 2 (εναλλακτική)

```
shared data[n], P,  // data array input/output
    index = 0;
    lock_type l;
```

```
main :
    P parallel tasks, t=0,1,2,...P-1
```

```
task(t) {
    while (my_index = getWork()) > -1) {
        result = f(data[my_index];
        putResult(result, my_index );
    }
```

Απλή δυναμική κατανομή 3 (εναλλακτική)

```
int getWork() {  
    Lock(l);  
    if (index > -1 && index_in < n-1)  
        index++;    // still data to process  
    else  
        index = -1; // no data to process  
    Unlock(l)  
    return index;  
}
```

```
PutResult(item, index) {    // save in same location  
                            // of data array  
    data[index] = item;  
}
```

Ζητήματα με τη δυναμική κατανομή 1

- Πολύ μικρά `blockSize` σημαίνει πολλά blocks, μεγαλύτερη πιθανότητα για καλύτερη εξισορρόπηση φορτίου.
- Πολλά blocks σημαίνει υπερβολική επιβάρυνση από τη διαχείριση των μηχανισμών ταυτοχρονισμού, δηλαδή κλειδωμάτων σε κάθε `getWork()`, `putResult()`.
- Γενικά απαιτείται πειραματισμός για να βρούμε το σωστό `blockSize`.

Ζητήματα με τη δυναμική κατανομή 2

Μικρό
BlockSize



Πολλά
Lock()
Unlock()

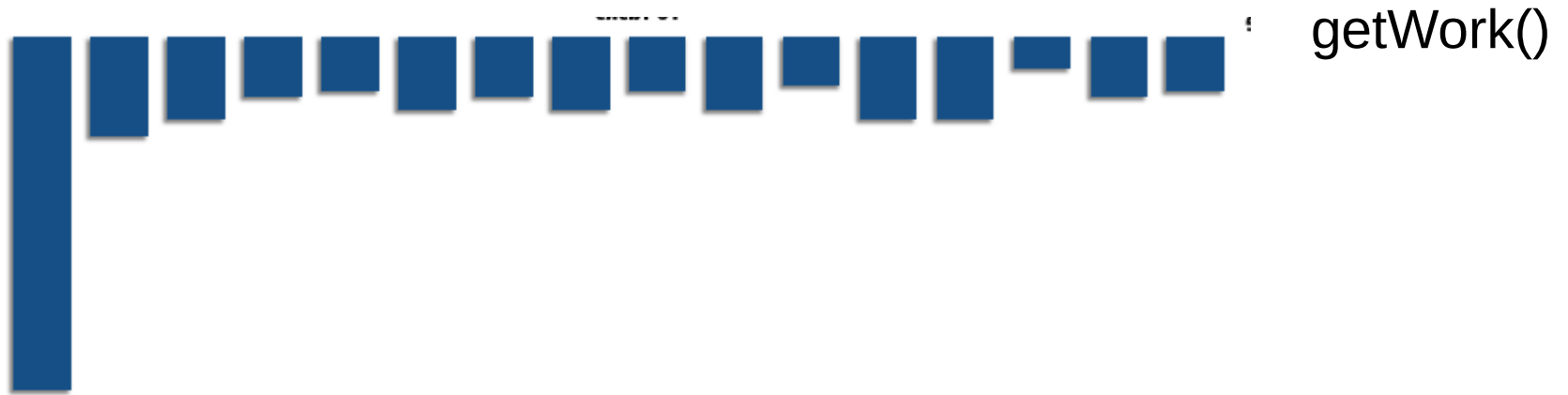
Μεγάλο
BlockSize



Λιγότερα
Lock()
Unlock()

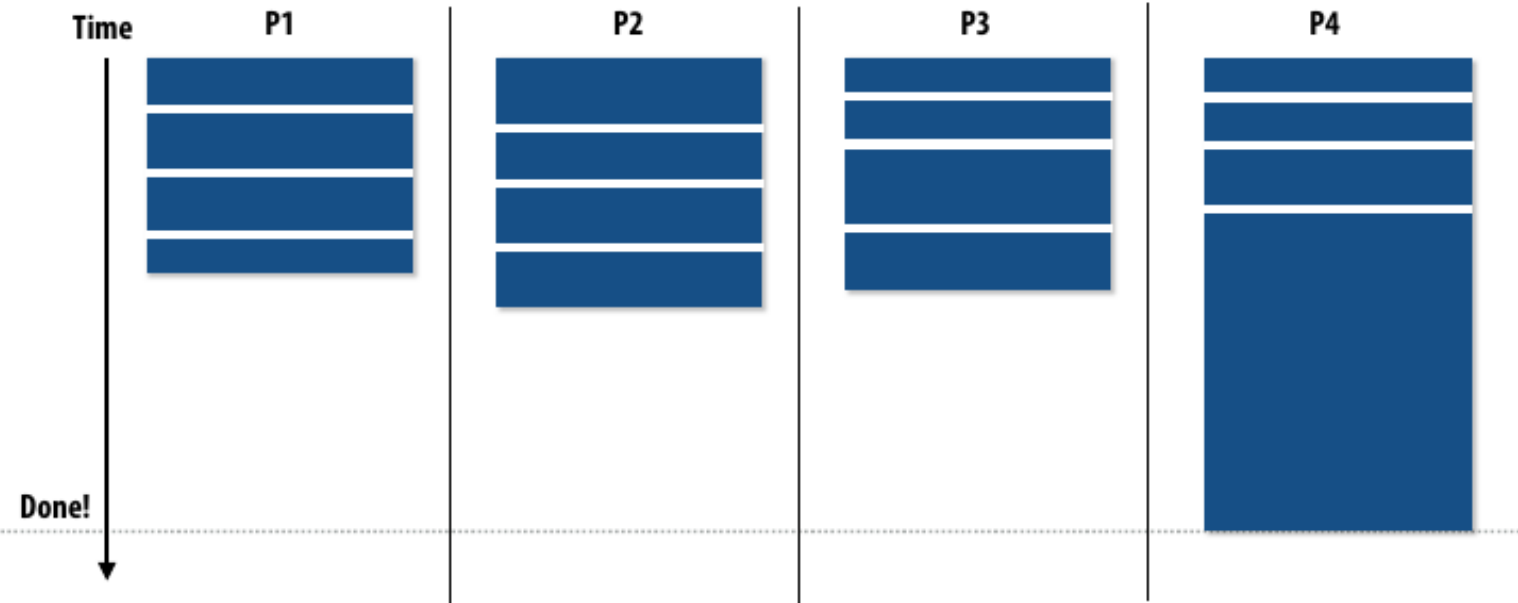
Επίδραση του blockSize στην επιβάρυνση ταυτοχρονισμού.

Μερικά ακόμη ζητήματα 1



- Έστω ότι η έχουμε 4 νήματα και μια ουρά με 16 εργασίες όπως παραπάνω.
- Τότε θα έχουμε ανισορροπία φορτίου από το τελευταίο στοιχείο ή εργασία.

Μερικά ακόμη ζητήματα 2

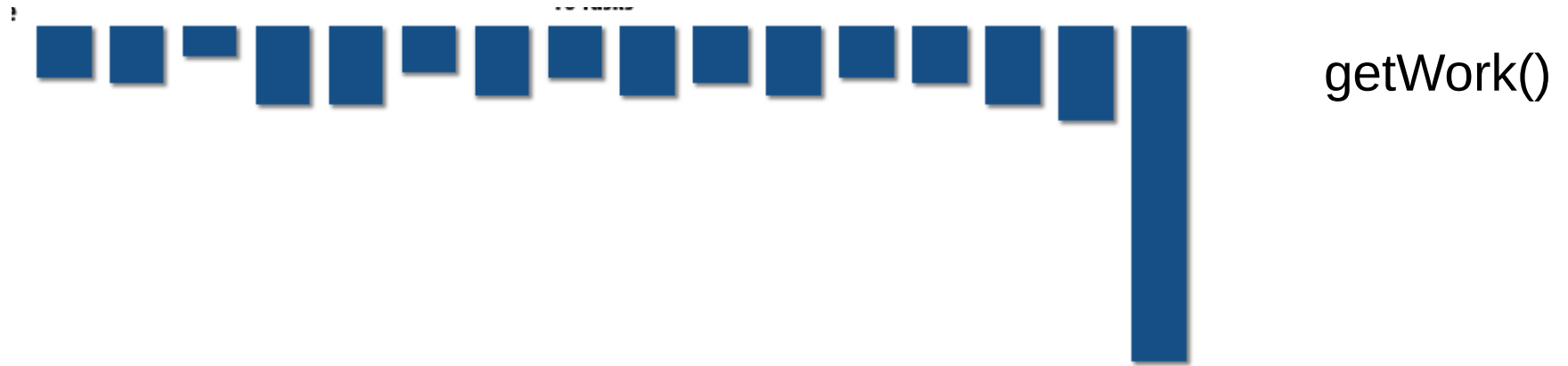


Ανισορροπία φορτίου λόγω της τελευταίας εργασίας.

Πιθανές λύσεις:

- Ανάλυση της τελευταίας εργασίας σε μικρότερες, αν είναι εφικτό.
- Εφαρμογή γενικά μικρότερου blockSize, με κόστος πιο πολλά Locks.

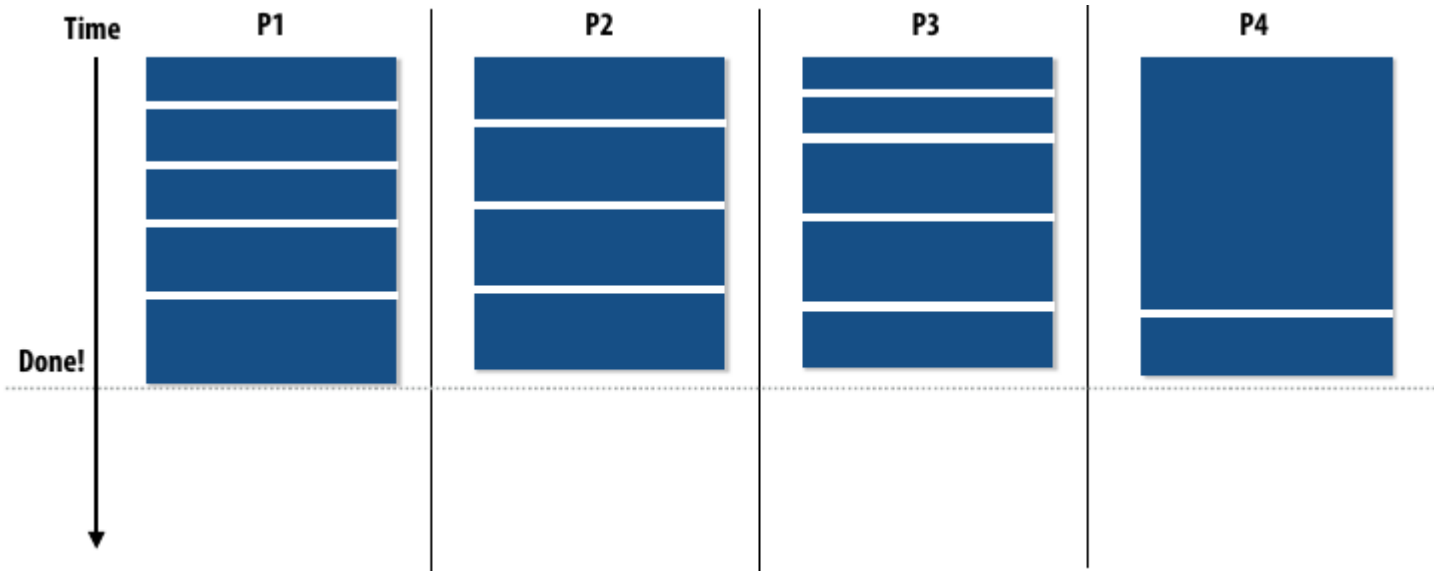
Μερικά ακόμη ζητήματα 3



Μια άλλη λύση

- Αναδιάταξη των εργασιών στην ουρά: πρώτα οι πολύ δαπανηρές εργασίες.

Μερικά ακόμη ζητήματα 4

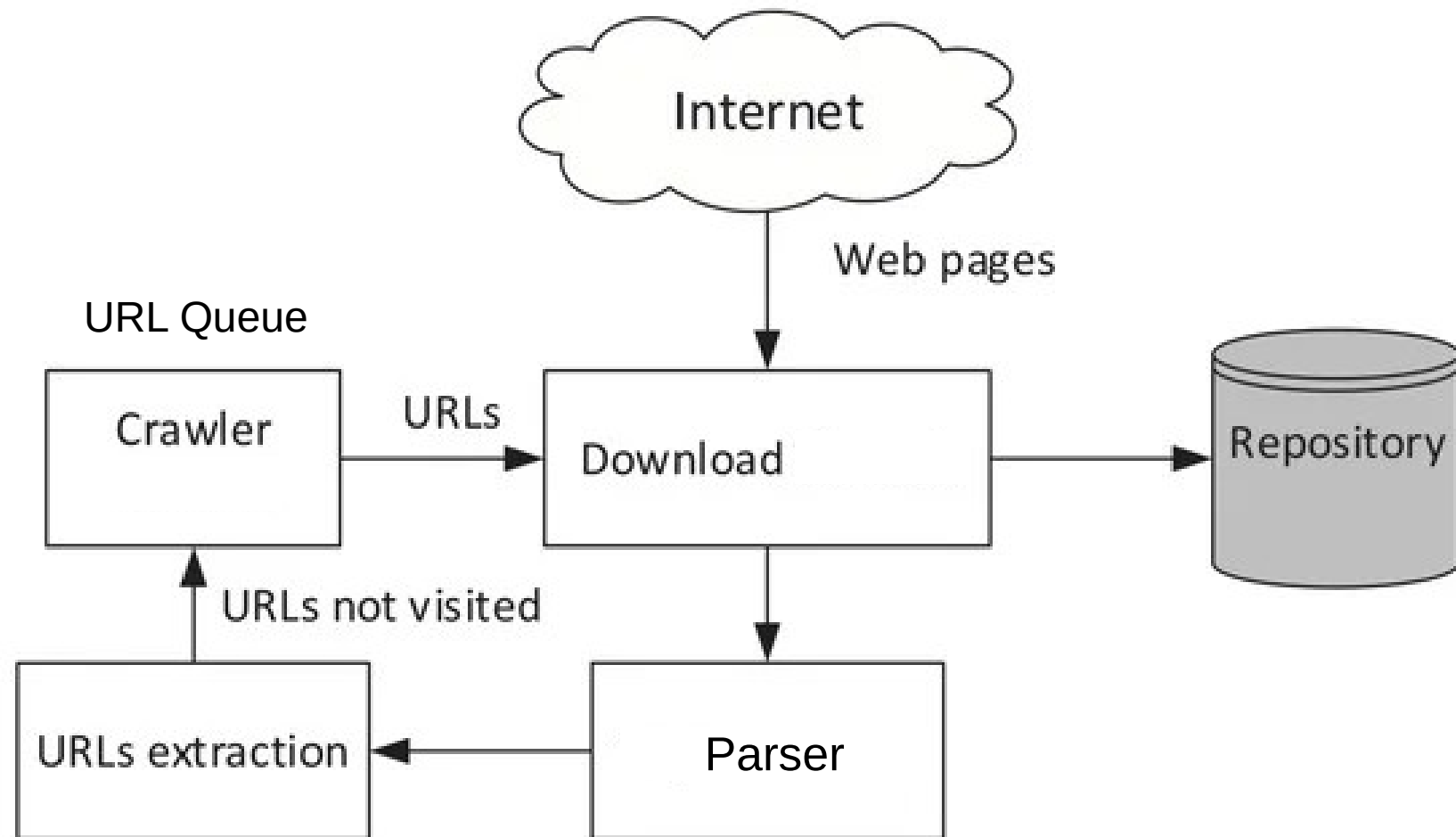


- Απαιτεί σχετική γνώση του υπολογιστικού φόρτου, πχ από προηγούμενες εκτελέσεις.
- Η τελευταία εργασία δεν θα πρέπει να εξαρτάται από προηγούμενες.

Γενίκευση δυναμικής κατανομής

- Το αποτέλεσμα που επιστρέφεται μπορεί να είναι νέο στοιχείο δεδομένων, το οποίο τοποθετείται στην ουρά της εισόδου.
- Παράδειγμα: crawler που σαρώνει ιστοσελίδες και επιστρέφει νέα links ιστοσελίδων.
- Πρέπει να τεθεί ένα όριο στην δημιουργία νέων δεδομένων, πχ το πλήθος των διαθέσιμων θέσεων στη λίστα με τα δεδομένα.
- Παράδειγμα: θέτουμε ένα όριο στο 'βάθος' της αναζήτησης του crawler.

Απλός Crawler



Κατανομή δεδομένων και κατανομή εργασιών

- Εναλλακτική μέθοδος κατανομής φόρτου.
- Δημιουργούμε εργασίες (tasks) στις οποίες κατανέμουμε δεδομένα.
- Δημιουργούμε έναν αριθμό νημάτων-εργαζομένων (worker threads), συνήθως πολύ μικρότερο από το πλήθος των tasks.
- Με χρήση δυναμικής κατανομής αναθέτουμε tasks σε νήματα-εργαζομένους προς εκτέλεση.
- Επιτρέπει τον σχεδιασμό παράλληλης λύσης υψηλού επιπέδου, ανεξάρτητης από τον ακριβή αριθμό των διαθέσιμων νημάτων-εργαζομένων.

Απλή δυναμική κατανομή εργασιών 1

shared $x[n]$, P , $Q \ll P$

Main:

task [Q] array of tasks, $q=0,1,2,\dots,m-1$
P threads, $p=0,1,\dots,P-1$

task[q] {

block = n / Q ;
start = $t * \text{block}$;
stop = start + block;
if ($t == Q-1$) stop = n ;

for ($j = \text{start}; j < \text{stop}; j++$)
 $x[j] = f(x[j])$;

}

Απλή δυναμική κατανομή εργασιών 2

```
thread(p) {  
    while (my_index = getWork()) > -1  
        run_in_thread (task[my_index]);  
}
```

```
getWork() {  
    Lock(l)  
    index ++  
    if (index > Q) return -1 //empty list  
    else return index-1  
    Unlock(l)  
}
```

Αυτόματη κατανομή υπολογιστικού φόρτου 1

Μηχανισμοί υψηλότερου επιπέδου. Επιτρέπουν:

- Κατανομή των εργασιών σε συγκεκριμένο (σταθερό, γενικά πεπερασμένο) αριθμό νημάτων.
- Συλλογή των αποτελεσμάτων των υπολογισμών των νημάτων σε δομές δεδομένων.
- Καθορισμό εξαρτήσεων μεταξύ εργασιών και δρομολόγηση των εργασιών με βάση εξαρτήσεις.
- Ο προγραμματιστής γράφει κώδικα για τις εργασίες αλλά η ανάθεση, δρομολόγηση, συλλογή αποτελεσμάτων γίνεται “αυτόματα”.

Αυτόματη κατανομή υπολογιστικού φόρτου 2

- Παραδείγματα

`omp_for`, `omp_task` (in, out) στο OpenMP (C,C++)

`executors`, `futures`, `streams` στη Java

`pool`, `executor`, `queue`, `events` στη Python

`parallel.for`, `streams` ... στη C#